DBMON-99: Best Practice Summary

> **Series:** DBMON | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 03/26/2026

Overview

This notebook consolidates every actionable best practice for Dynatrace database monitoring extracted from the DBMON series (notebooks 01 through 06). It covers SQL databases, NoSQL databases, caches, message brokers, query analysis, dashboards, and alerting. Each best practice is definitive: it specifies exactly what to configure, what threshold to set, and what priority to assign.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail enabled
OneAgent	Deployed on all application hosts making database, cache, and messaging calls
ActiveGate	Host-based ActiveGate for Extensions 2.0 (remote DB metrics)
Permissions	`storage:spans:read`, `storage:entities:read`, `storage:metrics:read`
Prior Reading	DBMON-01 through DBMON-06 for full context

1. OneAgent Instrumentation

#	Best Practice	Recommended Setting/Value	Priority	Category
1	Deploy OneAgent on every host that makes database calls	OneAgent installed on all application servers, not just database servers	**Critical**	Deployment
2	Enable deep code-level instrumentation	OneAgent auto-instruments JDBC, ADO.NET, and native database drivers by default; do not disable	**Critical**	Deployment
3	Verify `db.system` attribute is populated	Run `fetch spans, from:-1h | filter isNotNull(db.system) | summarize count(), by: {db.system}` — every expected technology must appear	**Critical**	Validation
4	Verify `db.statement` capture is enabled	Confirm normalized SQL/commands appear in `db.statement` field; if blank, check OneAgent deep monitoring settings	**Critical**	Validation
5	Confirm `db.namespace` is present	Run `filter isNotNull(db.namespace)` on spans; missing values indicate driver-level gaps	**Recommended**	Validation

#	Best Practice	Recommended Setting/Value	Priority	Category
6	Ensure `server.address` and `server.port` resolve	These fields must contain the actual database endpoint, not `localhost` or `127.0.0.1` when the DB is remote	**Recommended**	Validation

2. ActiveGate Extensions

#	Best Practice	Recommended Setting/Value	Priority	Category
7	Use host-based ActiveGate for Extensions 2.0	Must be host-based, not Kubernetes-based — K8s-based AG does not support Extensions 2.0	**Critical**	Deployment
8	Install PostgreSQL extension	Captures: connections, transactions/sec, tuple operations, table/index sizes, lock waits, replication lag	**Recommended**	Extension
9	Install MySQL extension	Captures: connections, queries/sec, buffer pool hit ratio, InnoDB row ops, slow queries	**Recommended**	Extension
10	Install MS SQL Server extension	Captures: batch requests/sec, page life expectancy, buffer cache hit ratio, deadlocks, wait stats	**Recommended**	Extension
11	Install Oracle extension	Captures: sessions, tablespace usage, SGA/PGA memory, redo log waits, library cache hit ratio	**Recommended**	Extension
12	Install MongoDB extension	Captures: connections, operations/sec, document metrics, replica set health, storage engine stats	**Recommended**	Extension
13	Set extension polling interval to 60 seconds	Default polling; do not exceed 300s or real-time visibility degrades	**Recommended**	Configuration

3. Span Attribute Coverage

#	Best Practice	Recommended Setting/Value	Priority	Category
14	Validate all six core DB span attributes	Every database span must carry: `db.system`, `db.statement`, `db.operation`, `db.namespace`, `server.address`, `server.port`	**Critical**	Data Quality
15	Check for NULL fields periodically	Run `filter isNotNull(db.operation)` — NULL means the driver is not reporting the operation type; investigate driver version	**Recommended**	Data Quality
16	Verify span.kind is CLIENT for outgoing DB calls	All database spans from the calling application must have `span.kind == "CLIENT"`	**Recommended**	Data Quality
17	For Kafka, verify `messaging.system` attribute	Kafka spans use `messaging.system` (not `db.system`); confirm `messaging.destination.name`, `messaging.operation`, and `messaging.kafka.consumer.group` are populated	**Critical**	Data Quality
18	For RabbitMQ, verify `messaging.system` attribute	RabbitMQ spans use `messaging.system` (not `db.system`); confirm `messaging.destination.name` is populated	**Critical**	Data Quality

4. SQL Database Monitoring

#	Best Practice	Recommended Setting/Value	Priority	Category
19	Set slow query threshold for SQL databases	**500ms** — any SQL query exceeding `duration > 500000000` (500ms in nanoseconds) is classified as slow	**Critical**	Threshold

#	Best Practice	Recommended Setting/Value	Priority	Category
20	Track read/write ratio	Classify operations: `SELECT` = READ, `INSERT/UPDATE/DELETE` = WRITE; monitor ratio shifts over time	**Recommended**	Analysis
21	Rank queries by total execution time, not average	Sort by `total_time_ms = sum(duration)` to find highest-impact patterns regardless of individual speed	**Critical**	Optimization
22	Monitor response time distribution using latency tiers	Bucket into: `<1ms`, `1–10ms`, `10–100ms`, `100ms–1s`, `>1s` — track tier distribution over time	**Recommended**	Analysis
23	Baseline P50, P95, P99 over 24 hours	Run hourly percentile query on `duration` for each `db.system`; this is your performance reference point	**Critical**	Baseline
24	Monitor vendor-specific patterns	PostgreSQL: vacuum/lock contention; MySQL: buffer pool/replication lag; MSSQL: page life expectancy/deadlocks; Oracle: tablespace/SGA	**Recommended**	Vendor
25	Track errors by `otel.status_message`	Group `otel.status_code == "ERROR"` by `otel.status_message` to distinguish connection timeouts, deadlocks, and constraint violations	**Critical**	Error Handling

5. NoSQL Database Monitoring

#	Best Practice	Recommended Setting/Value	Priority	Category
26	Set slow query threshold for NoSQL databases	**100ms** for MongoDB; **300ms** critical threshold for all NoSQL	**Critical**	Threshold

#	Best Practice	Recommended Setting/Value	Priority	Category
27	Monitor MongoDB by collection	Group by `db.mongodb.collection` in addition to `db.namespace` — collection-level granularity reveals hot spots	**Critical**	MongoDB
28	Track DynamoDB Scan-to-Query ratio	`Scan` operations read every item in the table; ratio must be Query-dominated; any high Scan count signals missing Global Secondary Indexes	**Critical**	DynamoDB
29	Set Cassandra slow threshold at 200ms	`filter duration > 200000000` — Cassandra operations exceeding 200ms indicate partition hotspots or cross-DC reads	**Recommended**	Cassandra
30	Monitor Cosmos DB by operation and error rate	Track `ReadItem`, `CreateItem`, `Query`, `ReplaceItem` separately; errors indicate RU throttling (HTTP 429)	**Recommended**	Cosmos DB
31	Classify NoSQL read/write operations correctly	READ: `find`, `Query`, `GetItem`, `SELECT`, `ReadItem`, `get`, `search`; everything else: WRITE	**Recommended**	Analysis
32	Run cross-database comparison	Compare `total_calls`, `avg_ms`, `p95_ms`, `error_rate_pct` across all NoSQL systems in a single query for unified health view	**Recommended**	Analysis

6. Cache Monitoring (Redis / Memcached)

#	Best Practice	Recommended Setting/Value	Priority	Category
33	Set Redis slow command threshold at 5ms	`filter duration > 5000000` — Redis operations should complete in microseconds; anything over 5ms is abnormal	**Critical**	Threshold
34	Monitor Redis read/write balance	READ: `GET`, `MGET`, `HGET`, `HGETALL`, `LRANGE`, `SMEMBERS`, `ZRANGE`; WRITE: `SET`, `MSET`, `HSET`, `LPUSH`, `RPUSH`, `SADD`, `ZADD`, `DEL`	**Recommended**	Analysis

#	Best Practice	Recommended Setting/Value	Priority	Category
35	Alert on Redis latency spikes	Track `p95_us` (microseconds) over time at 5m intervals; any sustained increase above 5000us (5ms) triggers investigation	**Critical**	Alerting
36	Watch for expensive Redis commands	`KEYS *`, `SMEMBERS` on large sets, `LRANGE` with large ranges cause latency spikes — flag any occurrence	**Critical**	Anti-Pattern
37	Report cache metrics in microseconds, not milliseconds	Redis and Memcached latency is sub-millisecond; use `avg(duration) / 1000.0` for microsecond precision	**Recommended**	Reporting

7. Message Broker Monitoring (Kafka / RabbitMQ)

#	Best Practice	Recommended Setting/Value	Priority	Category
38	Track Kafka throughput by topic	Group by `messaging.destination.name` and `messaging.operation` (`publish` vs `process`) at 5m intervals	**Critical**	Kafka
39	Monitor Kafka consumer group processing latency	Group by `messaging.kafka.consumer.group` and `messaging.destination.name` ; track `avg_ms` and `p95_ms` per group	**Critical**	Kafka
40	Detect Kafka consumer errors	Filter `otel.status_code == "ERROR"` on `messaging.operation == "process"` spans; any sustained errors indicate poisoned messages or deserialization failures	**Critical**	Kafka
41	Track RabbitMQ publish/consume rate per queue	Group by `messaging.destination.name` and `messaging.operation` ; a publish rate exceeding consume rate signals backpressure	**Critical**	RabbitMQ
42	Monitor RabbitMQ message throughput trend	Use `makeTimeseries` at 5m intervals grouped by `messaging.operation` to detect throughput drops	**Recommended**	RabbitMQ

8. Search Engine Monitoring (Elasticsearch)

#	Best Practice	Recommended Setting/Value	Priority	Category
43	Set Elasticsearch slow query threshold at 200ms	`filter duration > 200000000` for search operations; indexing operations use a separate threshold	**Recommended**	Threshold
44	Break down Elasticsearch operations by type	Track `db.operation` (search, index, bulk, delete) separately; search latency and indexing throughput have different SLOs	**Recommended**	Analysis
45	Include OpenSearch in all Elasticsearch queries	Always filter `in(db.system, {"elasticsearch", "opensearch"})` — treat both identically	**Recommended**	Compatibility

9. Query Analysis and Optimization

#	Best Practice	Recommended Setting/Value	Priority	Category
46	Detect N+1 queries	Group spans by `trace.id` and `db.statement`; any pattern with `calls_per_trace >= 10` is an N+1 candidate; >= 50 is confirmed N+1	**Critical**	Anti-Pattern
47	Fix N+1 by batching	Replace N individual queries with a single batch query using `IN` clauses or JOINs	**Critical**	Remediation
48	Detect missing indexes heuristically	SELECT queries with `call_count >= 50` AND `avg_ms > 10` AND high `stddev_ms / avg_ms` ratio are index candidates	**Recommended**	Optimization
49	Detect queries getting slower over time	Run 24h `makeTimeseries` of `avg(duration)` by `db.system` at 1h intervals; rising trend signals index degradation or table growth	**Recommended**	Trend

#	Best Practice	Recommended Setting/Value	Priority	Category
50	Flag dynamic SQL / poor parameterization	Count one-off query patterns (`call_count == 1`); high count per `db.system` means queries are not parameterized, preventing plan caching	**Recommended**	Anti-Pattern
51	Prioritize optimization by total time impact	Sort query patterns by `total_time_ms = sum(duration)` , not by `avg_ms` — a fast query called 1M times has more impact than a slow query called once	**Critical**	Optimization
52	Use the tail ratio to detect outliers	Calculate `p99_ms / p50_ms` ; a ratio above 10 indicates severe tail latency requiring investigation	**Recommended**	Analysis
53	Monitor connection pool pressure	Track `countIf(otel.status_code == "ERROR")` grouped by `dt.entity.service` and `server.address` ; connection refused or timeout errors indicate pool exhaustion	**Critical**	Connection Pool
54	Track error rate trend over 6h at 5m intervals	Use `makeTimeseries` with `total` and `errors` counts; a rising error trend within 30 minutes demands immediate action	**Critical**	Error Handling

10. Dashboards and KPIs

#	Best Practice	Recommended Setting/Value	Priority	Category
55	Build a unified health overview tile	Single query: group by `db.system` , show `total_calls` , `avg_ms` , `p95_ms` , `error_rate_pct` , `slow_rate_pct`	**Critical**	Dashboard
56	Include a "total database calls" single-value tile	`fetch spans, from:-1h | filter isNotNull(db.system) | summarize total_db_calls = count()`	**Critical**	Dashboard

#	Best Practice	Recommended Setting/Value	Priority	Category
57	Add "top 10 heaviest services" table tile	Group by `dt.entity.service` with `call_count`, `avg_ms`, `error_count`; resolve names with `entityName()`	**Critical**	Dashboard
58	Show P50 vs P95 vs P99 trend chart	6h timeseries at 5m intervals; three lines on one chart reveals tail latency divergence	**Critical**	Dashboard
59	Add read vs write ratio trend tile	Classify operations into READ/WRITE; 6h timeseries at 5m intervals	**Recommended**	Dashboard
60	Add queries-per-minute trend by db.system	6h timeseries at 1m intervals; used for throughput anomaly detection	**Recommended**	Dashboard
61	Add today-vs-yesterday volume comparison	Use `append` pattern: `from:-24h` for today, `from:-48h, to:-24h` for yesterday	**Recommended**	Dashboard
62	Include slow query detail table	Last 15m, `duration > 500000000`, fields: timestamp, db.system, db.namespace, db.statement, duration_ms, service_name	**Critical**	Dashboard

11. Alerting Thresholds

Error Rate Alerts

#	Best Practice	Recommended Setting/Value	Priority	Category
63	Alert on error rate > 5% for 5 minutes	Severity: **Critical** — immediate page	**Critical**	Alert
64	Alert on error rate > 1% for 15 minutes	Severity: **Warning** — notification	**Critical**	Alert
65	Alert on any connection refused error	Severity: **Critical** — immediate page; zero tolerance for connection failures	**Critical**	Alert
66	Alert on any deadlock detection	Severity: **Warning** — notification	**Recommended**	Alert

Slow Query Alerts (P95 Thresholds)

#	Best Practice	Recommended Setting/Value	Priority	Category
67	SQL Warning threshold	P95 > **200ms** over 5-minute window	**Critical**	Alert
68	SQL Critical threshold	P95 > **500ms** over 5-minute window	**Critical**	Alert
69	NoSQL Warning threshold	P95 > **100ms** over 5-minute window	**Critical**	Alert
70	NoSQL Critical threshold	P95 > **300ms** over 5-minute window	**Critical**	Alert
71	Cache (Redis/Memcached) Warning threshold	P95 > **5ms** over 5-minute window	**Critical**	Alert
72	Cache (Redis/Memcached) Critical threshold	P95 > **20ms** over 5-minute window	**Critical**	Alert
73	Elasticsearch Warning threshold	P95 > **500ms** over 5-minute window	**Recommended**	Alert
74	Elasticsearch Critical threshold	P95 > **2000ms** over 5-minute window	**Recommended**	Alert

Throughput Alerts

#	Best Practice	Recommended Setting/Value	Priority	Category
75	Alert on throughput drop > 50% vs 7-day average	Compare current queries/min to 7-day baseline; sudden drop indicates upstream failure or connectivity loss	**Critical**	Alert
76	Alert on slow query count > 10 per 5-minute window	Sustained slow query volume indicates systemic degradation, not a one-off outlier	**Recommended**	Alert

12. SLO Definitions

#	Best Practice	Recommended Setting/Value	Priority	Category
77	Define availability SLO	**99.9%** of database calls succeed (`otel.status_code != "ERROR"`) measured over 24h rolling window	**Critical**	SLO
78	Define latency SLO	**95%** of calls complete under the vendor-specific P95 threshold (SQL: 500ms, NoSQL: 300ms, Cache: 20ms, Search: 2000ms)	**Critical**	SLO
79	Define throughput SLO	No more than **50% drop** from 7-day baseline queries/min	**Recommended**	SLO
80	Measure SLO compliance hourly	Run `makeTimeseries` at 1h intervals with `total` and `errors` / `under_threshold` counts per `db.system`	**Critical**	SLO
81	Track SLO budget burn rate	If 24h availability drops below 99.9%, calculate remaining error budget: `(1 - 0.999) * total_calls - error_count`	**Recommended**	SLO

13. DQL Query Standards

#	Best Practice	Recommended Setting/Value	Priority	Category
82	Always specify a time range on fetch	`fetch spans, from:-1h` — never omit `from:` on span queries; default 2h scan is excessive	**Critical**	DQL
83	Filter on `isNotNull(db.system)` first	This is the primary filter to isolate database spans; apply immediately after `fetch`	**Critical**	DQL
84	Use named parameters in functions	`round(value, decimals: 2)` , `if(cond, then: "x", else: "y")` — positional parameters cause errors	**Critical**	DQL

#	Best Practice	Recommended Setting/Value	Priority	Category
85	Alias all aggregations	`summarize c = count()`, not `summarize count()` — unaliased aggregations cannot be used in `sort` or `fieldsAdd`	**Critical**	DQL
86	Convert duration to human-readable units	Span `duration` is in nanoseconds; divide by `1000.0` for microseconds, `1000000.0` for milliseconds	**Critical**	DQL
87	Use `entityName()` to resolve service IDs	`fieldsAdd service_name = entityName(dt.entity.service, type:"dt.entity.service")` — never show raw entity IDs in dashboards	**Recommended**	DQL
88	Filter early, sort last	Apply `filter` immediately after `fetch`; apply `sort` only after `summarize`; never `sort` right after `fetch`	**Critical**	DQL
89	Use `countIf()` for conditional aggregation	`errors = countIf(otel.status_code == "ERROR")` inside `summarize` — do not use separate filter + count	**Recommended**	DQL
90	Calculate error rate with safe division	`error_rate_pct = round((toDouble(error_count) / toDouble(total_calls)) * 100, decimals: 2)` — use `toDouble()` to avoid integer truncation	**Recommended**	DQL
